



CCF-GESP C++四级

编程能力等级认证



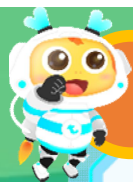


第二课

指针 2



01 指针和数组



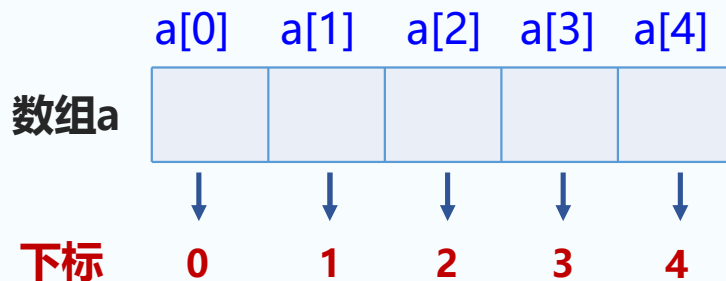
知识讲解

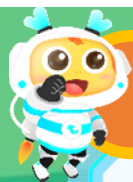
数组

数组就是一组数据，是由一组具有相同数据类型的变量组成的。

- 组成数组的变量称为数组元素，元素的本质是变量。
- 数组中的元素是有顺序的，每个元素都有对应的下标。
- 数组下标是一个整数，范围是[0, 长度-1]

```
int arr[5] = {1, 2, 3, 4, 5};
```





知识讲解

数组地址

数组名就是数组地址，也是**第一个元素的存储地址**，因此也称为**数组首地址**。

```
int arr[5] = {1, 2, 3, 4, 5};
```

/

数组地址

以下**三种方式**都可以获取到数组地址



```
cout<<arr<<endl;
```

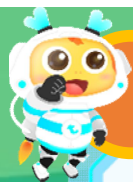
```
0xc8de5ff890
```

```
cout<<&arr<<endl;
```

```
0xc8de5ff890
```

```
cout<<&arr[0]<<endl;
```

```
0xc8de5ff890
```



知识讲解

指针与数组

指针变量也可以**指向**数组
需要使用和数组类型**相同**的指针变量

指针p指向
数组arr

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
int *p = arr;
```

```
cout<<*p<<endl;
```

1

```
*p = 100;
```

```
for(int i=0;i<5;i++)
```

```
    cout<<arr[i]<<" ";
```

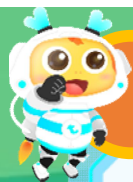
100 2 3 4 5

元素

下标

100	2	3	4	5
0	1	2	3	4

↑ 指向
p



知识讲解

直接使用数组指针访问元素

既然`arr`和`p`都是数组地址，也可以通过**指针p**访问数组元素

```
int arr[5] = {1, 2, 3, 4, 5};  
int *p = arr;  
cout<<p[1]<<endl; 2  
p[2]=150;  
for(int i=0;i<5;i++)  
    cout<<p[i]<<" ";
```

1 2 150 4 5

元素
下标

1	2	150	4	5
0	1	2	3	4

↑ 指向
p



知识讲解

指向数组元素的指针

```
int arr[5]={1,2,3,4,5};
```

```
int *p1=&arr[1];
```

```
*p1=arr[4];
```

元素	1	2	3	4	5
下标	0	1	2	3	4

↑ 指向
p1



真题解析



选择题

(2023年6月) 一个数组定义为 `int a[5] = {1, 2, 3, 4, 5};`, 一个指针定义为 `int * p = &a[2];`, 则执行 `a[1] = *p;`后, 数组 `a` 中的值会变为 (**A**)。

- A. {1,3,3,4,5}
- B. {2,2,3,4,5}
- C. {1,2,2,4,5}
- D. {1,2,3,4,5}



真题解析



选择题

(2023年9月) 一个数组定义为 `int a[5] = {1, 2, 3, 4, 5};` , 一个指针定义为 `int * p = &a[2];` , 则执行 `*p = a[1];` 后, 数组a中的值会变为 (**A**) 。

- A. {1, 2, 2, 4, 5}
- B. {1, 3, 3, 4, 5}
- C. {1, 2, 3, 3, 5}
- D. {1, 2, 4, 4, 5}



选择题

(2023年12月) 下面C++代码执行后输出是 (C)。

```
int arr[3]={1,2,3};  
int *p=NULL;  
p=arr;  
p++;  
cout<<*p<<endl;
```

- A. 1, 2, 3
- B. 1
- C. 2
- D. 3



知识讲解



指针变量和整数的运算

指针变量可以和一个**整数**进行**加法**、**减法**运算，一般配合数组进行使用。

- 指针变量**加1**，相当于加一个**存储单元**；
- 指针变量**减1**，相当于减一个**存储单元**；

```
int arr[5] = {1, 2, 3, 4, 5};
```

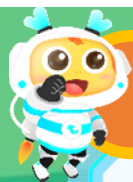
```
int *p = arr;
```

```
cout<<p<<endl;
```

```
cout<<p+1<<endl;
```

```
0x6b0a9ff8a0  
0x6b0a9ff8a4
```

} 4个字节



知识讲解

计算数组元素的地址

由于数组中的元素是按顺序**连续存放**在内存中，因此可以根据**数组地址**，计算出所有元素的地址。

```
int arr[5] = {1, 2, 3, 4, 5};  
int *p = arr;  
cout << *(p+1);
```

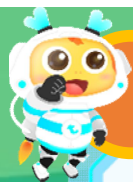
等价于 $p[1]$

*优先级高

元素	1	2	3	4	5
下标	0	1	2	3	4
元素地址	p	$p+1$	$p+2$	$p+3$	$p+4$

```
// 使用指针运算遍历数组  
for(int i=0;i<5;i++){  
    cout << *(p+i) << " ";  
}
```

```
// 数组名也可以  
for(int i=0;i<5;i++){  
    cout << *(arr+i) << " ";  
}
```



知识讲解

自增和自减运算

指针变量可以进行++和--运算，相当于加/减一个**存储单元**

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
int *p = arr;
```

```
cout<<*p++;
```

后++

1 优先级高

```
cout<<*p;
```

2

元素

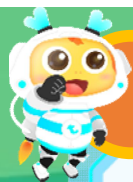
下标

1	2	3	4	5
0	1	2	3	4

元素地址

↑
p

↑
p++



知识讲解

自增和自减运算

指针变量可以进行++和--运算，相当于加/减一个**存储单元**

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
int *p = arr;
```

前++

```
cout << *++p;
```

2

```
cout << *p;
```

2

元素

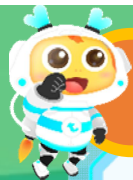
下标

1	2	3	4	5
0	1	2	3	4

元素地址

↑
p

↑
p++



知识讲解

自增和自减运算

指针变量可以进行++和--运算，相当于加/减一个**存储单元**

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
int *p = arr;
```

```
cout<<(*p)++;
```

1

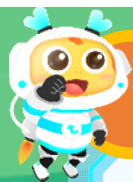
```
cout<<++(*p);
```

3

元素	3	2	3	4	5
下标	0	1	2	3	4

元素地址

↑
p



知识讲解

自增和自减运算

使用**指针变量**和++运算遍历数组

```
int arr[5] = {1, 2, 3, 4, 5};  
int *p = arr;
```

元素	1	2	3	4	5
下标	0	1	2	3	4
元素地址	p				

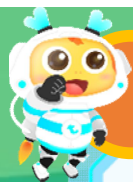
```
// 使用指针运算遍历数组  
for(int i=0;i<5;i++){  
    cout<<*p++<<" ";  
}
```

1 2 3 4 5

```
// 数组名可以吗?  
for(int i=0;i<5;i++){  
    cout<<*arr++<<" ";  
}
```



数组名是常量



知识讲解

指针变量之间的运算

两个指针变量只能进行**减法**运算，计算结果是两个指针之间**相差**多少个存储单元。

```
int arr[5] = {1, 2, 3, 4, 5};
```

```
int *p1=&arr[1];
```

```
int *p2=&arr[4];
```

```
cout<<p2-p1<<endl;
```

3

相差3个存储单元

元素
下标

1	2	3	4	5
0	1	2	3	4

↑
p1

↑
p2



课堂练习



选择题

阅读程序，写出程序的运行的结果(**D**)

```
int a[]={1,2,3,4,5,6};  
int *p=a;  
*(p+3)+=2;  
cout<<*p<<" "<<*(p+3)<<endl;
```

A. 0 5

B. 1 5

C. 0 6

D. 1 6



课堂练习

选择题

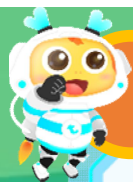
阅读程序，写出程序的运行的结果(**B**)

```
double b[10],*p1,*p2;
p1=b;
p2=&b[5];
cout<<p2-p1;
```

- A. 40
- B. 5
- C. 6
- D. 不确定



03 结构体和指针



知识讲解



结构体

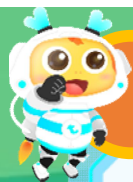
结构体是由多个类型不同（或相同）的数据组合而成的**数据类型**。

成员变量

```
struct 结构体名{  
    数据类型1 变量名1;  
    数据类型2 变量名2;  
    .....  
};
```

```
struct stu{  
    int id;  
    string name;  
    double score;  
};
```

```
stu a={1,"yaya",98};  
a.score=95;  
cout<<a.score<<endl;
```



知识讲解

结构体指针



指向结构体类型变量的**指针**，称为**结构体指针**

```
struct stu{  
    int id;  
    string name;  
    double score;  
};
```

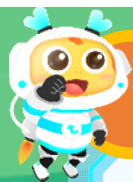
创建结构体变量a并初始化

创建指针变量并初始化

```
stu a={1,"yaya",98.5};  
stu *p = &a;
```



学号: 1
姓名: yaya
分数: 98.5



知识讲解

结构体指针

```
stu a={1,"yaya",98.5};  
stu *p = &a;
```

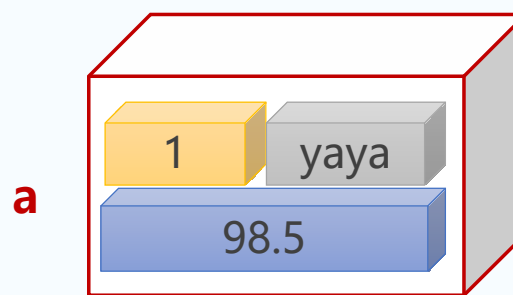
通过**结构体指针**访问结构体变量中的数据

- 使用**指针运算符***访问变量a
- 再使用**点**访问成员变量

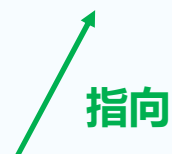
访问变量a

(*)p).id;

*优先级低



0x73fd3ff998

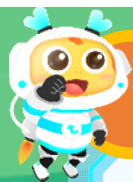


指向

p

0x73fd3ff998

0x73fd3ff990



知识讲解

直接访问结构体成员变量



语法格式： **指针名->成员变量名**

```
struct stu{  
    int id;  
    string name;  
    double score;  
};
```

```
stu a={1,"yaya",98.5};
```

```
stu *p = &a;
```

```
cout<< p->id <<endl;
```

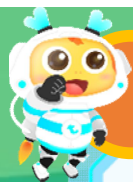
1

```
cout<< p->name <<endl;
```

yaya

```
cout<< p->score <<endl;
```

98.5



知识讲解



通过指针修改成员变量



使用**=赋值运算符**来进行赋值和修改

```
// 创建结构体类型stu
struct stu{
    int id;
    string name;
    double score;
};

stu a={1,"yaya",98.5};
stu *p = &a;

p->score =100;

cout<< p->score <<endl;
```

修改成员变量



学号: 1
姓名: yaya
分数: 100

分数: 100

100



选择题

(2023年9月) 下列关于C++语言中指针的叙述, **不正确**的是 (**D**)。

- A. 可以定义指向 int 类型的指针。
- B. 可以定义指向自定义结构体类型的指针。
- C. 自定义结构体类型可以包含指针类型的元素。
- D. 不能定义指向void类型的指针, 那没有意义。



课堂练习

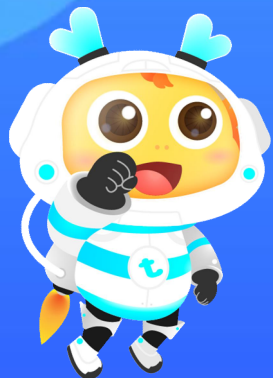


选择题

阅读以下程序，想要创建结构体指针变量，横线应该填写的是（ **B** ）

```
struct node{  
    int id;  
    string name;  
    double score;  
};  
node a;  
____ *p = &a;
```

- A. int
- B. node
- C. double
- D. string



感谢你的学习